Java Array Quiz – Ausführliche Analyse der 20 Fragen

Benjamin Malicke ©

26. Oktober 2025

Inhaltsverzeichnis

1 Einleitung **3**

2 Frage 0 **3**
2.1 Code . . . 3
2.2 Initialisierung . . . 4
2.3 Schritt-für-Schritt-Ausführung . . . 4
2.4 Ausgabe . . . 4
2.5 Zusätzliche Erklärungen . . . 4

3 Frage 1 **4**
3.1 Code . . . 4
3.2 Initialisierung . . . 4
3.3 Schritt-für-Schritt-Ausführung . . . 4
3.4 Ausgabe . . . 4
3.5 Zusätzliche Erklärungen . . . 5

4 Frage 2 **5**
4.1 Code . . . 5
4.2 Initialisierung . . . 5
4.3 Schritt-für-Schritt-Ausführung . . . 5
4.4 Ausgabe . . . 5
4.5 Zusätzliche Erklärungen . . . 5

5 Frage 3 **5**
5.1 Code . . . 5
5.2 Initialisierung . . . 5
5.3 Schritt-für-Schritt-Ausführung . . . 5
5.4 Ausgabe . . . 5
5.5 Zusätzliche Erklärungen . . . 6

6 Frage 4 **6**
6.1 Code . . . 6
6.2 Initialisierung . . . 6
6.3 Schritt-für-Schritt-Ausführung . . . 6
6.4 Ausgabe . . . 6
6.5 Zusätzliche Erklärungen . . . 6

7 Frage 5 **6**
7.1 Code . . . 6
7.2 Initialisierung . . . 6
7.3 Schritt-für-Schritt-Ausführung . . . 6
7.4 Ausgabe . . . 7
7.5 Zusätzliche Erklärungen . . . 7

8 Frage 6 **7**
8.1 Code . . . 7
8.2 Initialisierung . . . 7
8.3 Schritt-für-Schritt-Ausführung . . . 7
8.4 Ausgabe . . . 7
8.5 Zusätzliche Erklärungen . . . 7

9 Frage 7 **7**
9.1 Code . . . 7
9.2 Initialisierung . . . 7
9.3 Schritt-für-Schritt-Ausführung . . . 7
9.4 Ausgabe . . . 7
9.5 Zusätzliche Erklärungen . . . 8

10 Frage 8 **8**
10.1 Code . . . 8
10.2 Initialisierung . . . 8
10.3 Schritt-für-Schritt-Ausführung . . . 8
10.4 Ausgabe . . . 8
10.5 Zusätzliche Erklärungen . . . 8

11 Frage 9 **8**
11.1 Code . . . 8
11.2 Initialisierung . . . 8
11.3 Schritt-für-Schritt-Ausführung . . . 8
11.4 Ausgabe . . . 8
11.5 Zusätzliche Erklärungen . . . 8

12 Frage 10 **9**
12.1 Code . . . 9
12.2 Initialisierung . . . 9
12.3 Schritt-für-Schritt-Ausführung . . . 9
12.4 Ausgabe . . . 9
12.5 Zusätzliche Erklärungen . . . 9

13 Frage 11 **9**
13.1 Code . . . 9
13.2 Initialisierung . . . 9
13.3 Schritt-für-Schritt-Ausführung . . . 9
13.4 Ausgabe . . . 9
13.5 Zusätzliche Erklärungen . . . 9

14 Frage 12 **10**
14.1 Code . . . 10
14.2 Initialisierung . . . 10
14.3 Schritt-für-Schritt-Ausführung . . . 10
14.4 Ausgabe . . . 10
14.5 Zusätzliche Erklärungen . . . 10

15 Frage 13 **10**
15.1 Code . . . 10
15.2 Initialisierung . . . 10
15.3 Schritt-für-Schritt-Ausführung . . . 10
15.4 Ausgabe . . . 10
15.5 Zusätzliche Erklärungen . . . 10

16 Frage 14 **11**
16.1 Code . . . 11
16.2 Initialisierung . . . 11
16.3 Schritt-für-Schritt-Ausführung . . . 11
16.4 Ausgabe . . . 11
16.5 Zusätzliche Erklärungen . . . 11

17 Frage 15 **11**
17.1 Code . . . 11
17.2 Initialisierung . . . 11
17.3 Schritt-für-Schritt-Ausführung . . . 11
17.4 Ausgabe . . . 11
17.5 Zusätzliche Erklärungen . . . 11

18 Frage 16 **12**
18.1 Code . . . 12
18.2 Initialisierung . . . 12
18.3 Schritt-für-Schritt-Ausführung . . . 12
18.4 Ausgabe . . . 12
18.5 Zusätzliche Erklärungen . . . 12

19 Frage 17 **12**
19.1 Code . . . 12
19.2 Initialisierung . . . 12
19.3 Schritt-für-Schritt-Ausführung . . . 12
19.4 Ausgabe . . . 12
19.5 Zusätzliche Erklärungen . . . 12

20 Frage 18 **13**
20.1 Code . . . 13
20.2 Initialisierung . . . 13
20.3 Schritt-für-Schritt-Ausführung . . . 13
20.4 Ausgabe . . . 13
20.5 Zusätzliche Erklärungen . . . 13

21 Frage 19 **13**
21.1 Code . . . 13
21.2 Initialisierung . . . 13
21.3 Schritt-für-Schritt-Ausführung . . . 13
21.4 Ausgabe . . . 13
21.5 Zusätzliche Erklärungen . . . 14

1 Einleitung

Dieses Dokument analysiert und erklärt jede der 20 Fragen aus dem Java Array Quiz. Jede Frage enthält ein Java-Code-Snippet, das mit Arrays arbeitet. Wir werden den Code darstellen, eine schrittweise Ausführung (Dry-Run) durchführen und die Ergebnisse detailliert erläutern. Alle Erklärungen sind auf Deutsch und sehr ausführlich, um ein tiefes Verständnis zu fördern.

Die Fragen sind von 0 bis 19 nummeriert. Für jede Frage:

- **Code:** Der vollständige Java-Code.
- **Initialisierung:** Beschreibung des anfänglichen Zustands.
- **Schritt-für-Schritt-Ausführung:** Detaillierte Analyse jeder Operation.
- **Ausgabe:** Das Endergebnis und warum es so ist.
- **Zusätzliche Erklärungen:** Hinweise zu Konzepten wie Array-Indizes, Schleifen, Operatoren usw.

2 Frage 0

2.1 Code

```
int[] arr = {7, 3, 5, 9};
int res = arr[0] - arr[1] + arr[2] * arr[3];
System.out.println(res);
```

2.2 Initialisierung

Das Array `arr` wird mit den Werten {7, 3, 5, 9} initialisiert. Die Indizes sind: arr[0] = 7, arr[1] = 3, arr[2] = 5, arr[3] = 9. Eine Variable `res` wird berechnet.

2.3 Schritt-für-Schritt-Ausführung

1. Der Ausdruck `res = arr[0] - arr[1] + arr[2] * arr[3]` wird ausgewertet.
2. Zuerst die Multiplikation (aufgrund der Operatorpriorität: * vor + und -): arr[2] * arr[3] = 5 * 9 = 45.
3. Dann von links nach rechts: arr[0] - arr[1] = 7 - 3 = 4.
4. Anschließend: 4 + 45 = 49.
5. `res` erhält den Wert 49.
6. `System.out.println(res);` gibt 49 aus.

2.4 Ausgabe

Ausgabe: 49

2.5 Zusätzliche Erklärungen

In Java haben Operatoren Prioritäten: Multiplikation und Division vor Addition und Subtraktion. Ohne Klammern wird der Ausdruck von links nach rechts ausgewertet, aber mit Berücksichtigung der Prioritäten. Arrays sind nullbasiert, d.h. der erste Index ist 0. Hier gibt es keine Schleifen, nur eine einfache Berechnung.

3 Frage 1

3.1 Code

```java
int[] arr = {12, 14, 16, 28};
for (int i = 0; i < arr.length; i++) {
  arr[3] -= arr[i];
}
System.out.println(arr[3]);
```

3.2 Initialisierung

Array `arr` = {12, 14, 16, 28}. Indizes: arr[0]=12, arr[1]=14, arr[2]=16, arr[3]=28. Schleife läuft von i=0 bis i<4 (arr.length=4).

3.3 Schritt-für-Schritt-Ausführung

1. i=0: arr[3] -= arr[0] → 28 - 12 = 16. Nun arr[3]=16.
2. i=1: arr[3] -= arr[1] → 16 - 14 = 2. Nun arr[3]=2.
3. i=2: arr[3] -= arr[2] → 2 - 16 = -14. Nun arr[3]=-14.
4. i=3: arr[3] -= arr[3] → -14 - (-14) = 0. Nun arr[3]=0.
5. Schleife endet.
6. `System.out.println(arr[3]);` gibt 0 aus.

3.4 Ausgabe

Ausgabe: 0

3.5 Zusätzliche Erklärungen

Die Schleife modifiziert arr[3] in jedem Durchlauf, einschließlich wenn i=3, wo es sich selbst subtrahiert. Das ist erlaubt, da Arrays mutable sind. Beachte, dass arr[i] beim Subtrahieren den aktuellen Wert verwendet, der sich ändert.

4 Frage 2

4.1 Code

```
int[] arr = {1, 3, 5, 7};
for (int i = 1; i < arr.length; i+=2) {
  System.out.print(arr[i]);
}
```

4.2 Initialisierung

Array `arr` = {1, 3, 5, 7}. Schleife startet bei i=1, inkrementiert um 2, bis i<4.

4.3 Schritt-für-Schritt-Ausführung

1. i=1: print arr[1] = 3.
2. i += 2 → i=3.
3. i=3: print arr[3] = 7.
4. i += 2 → i=5, was >4 ist → Schleife endet.

4.4 Ausgabe

Ausgabe: 37 (ohne Leerzeichen, da print verwendet wird).

4.5 Zusätzliche Erklärungen

Die Schleife springt über ungerade Indizes (1,3). arr.length gibt 4 zurück. print() fügt keine Zeilenumbrüche hinzu, nur println() tut das.

5 Frage 3

5.1 Code

```
int[] arr = {10, 20, 30, 40};
System.out.println(arr[arr[0]/10]);
```

5.2 Initialisierung

Array `arr` = {10, 20, 30, 40}.

5.3 Schritt-für-Schritt-Ausführung

1. arr[0]/10 = 10/10 = 1 (Integer-Division).
2. arr[1] = 20.
3. println(20).

5.4 Ausgabe

Ausgabe: 20

5.5 Zusätzliche Erklärungen

Array-Indizes können aus Ausdrücken berechnet werden, solange sie ganzzahlig und im Bereich 0 bis length-1 sind. Hier ist 10/10=1, was gültig ist. Bei ungültigen Indizes gäbe es ArrayIndexOutOfBoundsException.

6 Frage 4

6.1 Code

```
int[] arr = {2, 4, 6, 8};
int sum = 0;
for (int i = 0; i < arr.length; i++) {
  if (arr[i] % 4 == 0) sum += arr[i];
}
System.out.println(sum);
```

6.2 Initialisierung

Array `arr` = {2, 4, 6, 8}, sum=0.

6.3 Schritt-für-Schritt-Ausführung

1. i=0: $2 \% 4 = 2 \neq 0 \rightarrow$ `skip`.
2. i=1: $4 \% 4 = 0 \rightarrow \text{sum} = 0 + 4 = 4$.
3. i=2: $6 \% 4 = 2 \neq 0 \rightarrow$ `skip`.
4. i=3: $8 \% 4 = 0 \rightarrow \text{sum} = 4 + 8 = 12$.

6.4 Ausgabe

Ausgabe: 12

6.5 Zusätzliche Erklärungen

Der Modulo-Operator % prüft Rest bei Division. Hier summiert es Vielfache von 4. If-Bedingungen filtern Elemente.

7 Frage 5

7.1 Code

```
int[] arr = {0, 1, 2, 3};
for (int i = arr.length - 1; i >= 0; i--) {
  System.out.print(arr[i]);
}
```

7.2 Initialisierung

Array `arr` = {0, 1, 2, 3}, i=3 (length-1=3).

7.3 Schritt-für-Schritt-Ausführung

1. i=3: print 3.
2. i-- $\rightarrow$ i=2: print 2.
3. i=1: print 1.
4. i=0: print 0.

7.4 Ausgabe

Ausgabe: 3210

7.5 Zusätzliche Erklärungen

Rückwärtsschleife zur Umkehrung der Ausgabe. i >= 0 inkludiert i=0.

8 Frage 6

8.1 Code

```
int[] arr = {1, 2, 3, 4};
System.out.println(arr[1] * arr[3]);
```

8.2 Initialisierung

Array `arr` = {1, 2, 3, 4}.

8.3 Schritt-für-Schritt-Ausführung

1. arr[1] = 2, arr[3] = 4.
2. 2 * 4 = 8.
3. println(8).

8.4 Ausgabe

Ausgabe: 8

8.5 Zusätzliche Erklärungen

Einfache Multiplikation zweier Array-Elemente. Keine Schleife nötig.

9 Frage 7

9.1 Code

```
int[] arr = {2, 4, 8};
for (int i = 0; i < arr.length; i++) {
  arr[i] = arr[i] / 2;
}
System.out.println(arr[2]);
```

9.2 Initialisierung

Array `arr` = {2, 4, 8}.

9.3 Schritt-für-Schritt-Ausführung

1. i=0: 2 / 2 = 1 → arr[0]=1.
2. i=1: 4 / 2 = 2 → arr[1]=2.
3. i=2: 8 / 2 = 4 → arr[2]=4.

9.4 Ausgabe

Ausgabe: 4

9.5 Zusätzliche Erklärungen

Integer-Division: / für Ganzzahlen rundet ab. Array wird in-place modifiziert.

10 Frage 8

10.1 Code

```
int[] arr = {5, 7, 9};
arr[2] = arr[0] + arr[1];
System.out.println(arr[2]);
```

10.2 Initialisierung

Array `arr` = {5, 7, 9}.

10.3 Schritt-für-Schritt-Ausführung

1. arr[2] = 5 + 7 = 12.
2. println(12).

10.4 Ausgabe

Ausgabe: 12

10.5 Zusätzliche Erklärungen

Überschreiben eines Elements mit Summe anderer. Der alte Wert von arr[2] (9) wird ignoriert.

11 Frage 9

11.1 Code

```
int[] arr = {1, 2, 3, 4, 5};
for (int i = 0; i < arr.length; i += 2) {
  System.out.print(arr[i]);
}
```

11.2 Initialisierung

Array `arr` = {1, 2, 3, 4, 5}, i=0.

11.3 Schritt-für-Schritt-Ausführung

1. i=0: print 1.
2. i+=2 → i=2: print 3.
3. i=4: print 5.
4. i=6 >5 → Ende.

11.4 Ausgabe

Ausgabe: 135

11.5 Zusätzliche Erklärungen

Schleife über gerade Indizes (0,2,4). Gibt die ungeraden Zahlen aus (1,3,5), da das Array so gefüllt ist.

12 Frage 10

12.1 Code

```
int[] arr = {4, 8, 12};
int res = 1;
for (int i = 0; i < arr.length; i++) {
  res *= arr[i] / 4;
}
System.out.println(res);
```

12.2 Initialisierung

Array `arr` = {4, 8, 12}, res=1.

12.3 Schritt-für-Schritt-Ausführung

1. i=0: res *= (4/4) = 1 * 1 = 1.
2. i=1: res *= (8/4) = 1 * 2 = 2.
3. i=2: res *= (12/4) = 2 * 3 = 6.

12.4 Ausgabe

Ausgabe: 6

12.5 Zusätzliche Erklärungen

Produkt der (Element / 4). Integer-Division: 4/4=1, 8/4=2, 12/4=3.

13 Frage 11

13.1 Code

```
int[] arr = {2, 4, 8, 16};
System.out.println(arr[arr[1]/2]);
```

13.2 Initialisierung

Array `arr` = {2, 4, 8, 16}.

13.3 Schritt-für-Schritt-Ausführung

1. arr[1]/2 = 4/2 = 2.
2. arr[2] = 8.
3. println(8).

13.4 Ausgabe

Ausgabe: 8

13.5 Zusätzliche Erklärungen

Ähnlich wie Frage 3: Berechneter Index.

14 Frage 12

14.1 Code

```
int[] arr = {3, 6, 9};
for (int i = 0; i < arr.length; i++) {
  arr[0] += arr[i];
}
System.out.println(arr[0]);
```

14.2 Initialisierung

Array `arr` = {3, 6, 9}.

14.3 Schritt-für-Schritt-Ausführung

1. i=0: arr[0] += arr[0] = 3 + 3 = 6. Nun arr={6,6,9}.
2. i=1: arr[0] += arr[1] = 6 + 6 = 12. Nun arr={12,6,9}.
3. i=2: arr[0] += arr[2] = 12 + 9 = 21. Nun arr={21,6,9}.

14.4 Ausgabe

Ausgabe: 21

14.5 Zusätzliche Erklärungen

Beim Addieren ändert sich arr[0], und bei i=1 wird der alte arr[1] verwendet, der nicht geändert wurde. Aber arr[0] wächst.

15 Frage 13

15.1 Code

```
int[] arr = {1, 2, 3, 4};
for (int i = arr.length - 1; i >= 0; i -= 2) {
  System.out.print(arr[i]);
}
```

15.2 Initialisierung

Array `arr` = {1, 2, 3, 4}, i=3.

15.3 Schritt-für-Schritt-Ausführung

1. i=3: print 4.
2. i-=2 → i=1: print 2.
3. i-=2 → i=-1 <0 → Ende.

15.4 Ausgabe

Ausgabe: 42

15.5 Zusätzliche Erklärungen

Rückwärtsschleife mit Schritt -2: Indizes 3,1.

16 Frage 14

16.1 Code

```
int[] arr = {1, 3, 5, 7};
int sum = 0;
for (int i = 0; i < arr.length; i++) {
  if (arr[i] % 2 == 1) sum += arr[i];
}
System.out.println(sum);
```

16.2 Initialisierung

Array `arr` = {1, 3, 5, 7}, sum=0.

16.3 Schritt-für-Schritt-Ausführung

1. i=0: 1%2=1 → sum=1.
2. i=1: 3%2=1 → sum=4.
3. i=2: 5%2=1 → sum=9.
4. i=3: 7%2=1 → sum=16.

16.4 Ausgabe

Ausgabe: 16

16.5 Zusätzliche Erklärungen

Summiert ungerade Zahlen (alle hier).

17 Frage 15

17.1 Code

```
int[] arr = {5, 10, 15};
for (int i = 0; i < arr.length; i++) {
  arr[i] -= 5;
}
System.out.println(arr[2]);
```

17.2 Initialisierung

Array `arr` = {5, 10, 15}.

17.3 Schritt-für-Schritt-Ausführung

1. i=0: arr[0]=0.
2. i=1: arr[1]=5.
3. i=2: arr[2]=10.

17.4 Ausgabe

Ausgabe: 10

17.5 Zusätzliche Erklärungen

Subtrahiert 5 von jedem Element.

18 Frage 16

18.1 Code

```
int[] arr = {2, 3, 5, 7};
int sum = 0;
for (int i = 0; i < arr.length; i++) {
  if (arr[i] > 4) sum += arr[i];
}
System.out.println(sum);
```

18.2 Initialisierung

Array `arr` = {2, 3, 5, 7}, sum=0.

18.3 Schritt-für-Schritt-Ausführung

1. i=0: 2>4? Nein.
2. i=1: 3>4? Nein.
3. i=2: 5>4 → sum=5.
4. i=3: 7>4 → sum=12.

18.4 Ausgabe

Ausgabe: 12

18.5 Zusätzliche Erklärungen

Filtert und summiert Elemente größer als 4.

19 Frage 17

19.1 Code

```
int[] arr = {1, 2, 3, 4};
for (int i = 0; i < arr.length; i++) {
  System.out.print(arr[arr.length - 1 - i]);
}
```

19.2 Initialisierung

Array `arr` = {1, 2, 3, 4}.

19.3 Schritt-für-Schritt-Ausführung

1. i=0: arr[4-1-0]=arr[3]=4.
2. i=1: arr[4-1-1]=arr[2]=3.
3. i=2: arr[4-1-2]=arr[1]=2.
4. i=3: arr[4-1-3]=arr[0]=1.

19.4 Ausgabe

Ausgabe: 4321

19.5 Zusätzliche Erklärungen

Berechnet umgekehrte Indizes in der Schleife, um das Array rückwärts auszugeben.

20 Frage 18

20.1 Code

```
int[] arr = {1, 2, 3, 4};
for (int i = 0; i < arr.length; i++) {
  arr[2] += i;
}
System.out.println(arr[2]);
```

20.2 Initialisierung

Array `arr` = {1, 2, 3, 4}.

20.3 Schritt-für-Schritt-Ausführung

1. i=0: arr[2] += 0 = 3.
2. i=1: arr[2] += 1 = 4.
3. i=2: arr[2] += 2 = 6.
4. i=3: arr[2] += 3 = 9.

20.4 Ausgabe

Ausgabe: 9

20.5 Zusätzliche Erklärungen

Addiert die Schleifenvariable i zu arr[2] in jedem Durchlauf. Summe von 0 bis 3 ist 6, plus ursprünglicher 3 ergibt 9.

21 Frage 19

21.1 Code

```
int[] arr = {1, 2, 3};
for (int i = 0; i < arr.length; i++) {
  System.out.print(--arr[i] * arr[i]--);
}
```

21.2 Initialisierung

Array `arr` = {1, 2, 3}.

21.3 Schritt-für-Schritt-Ausführung

1. i=0: --arr[0] = 1-1=0 (pre-decrement), dann arr[0]-- = 0 (post-decrement, verwendet 0, dann arr[0]=-1). 0 * 0 = 0. Print 0.
2. i=1: --arr[1] = 2-1=1, arr[1]-- = 1, dann arr[1]=0. 1 * 1 = 1. Print 1.
3. i=2: --arr[2] = 3-1=2, arr[2]-- = 2, dann arr[2]=1. 2 * 2 = 4. Print 4.

21.4 Ausgabe

Ausgabe: 014

21.5 Zusätzliche Erklärungen

Pre-decrement (--vor) ändert und verwendet den neuen Wert. Post-decrement (nach--) verwendet den alten Wert und ändert danach. Der Multiplikationsausdruck wertet von links nach rechts aus. Das ist trickreich und zeigt die Reihenfolge der Operationen.